SYSTEM DESIGN, FROM SCRATCH · PART 1 / 4

The Anatomy of a Request

Every system that serves millions still answers to one request at a time. This is the companion reference for Part 1 — a quick-scan version of the post, with the same diagrams, simplified for print.

00

Why bother understanding this at all?

"System design" usually shows up first as an interview format, which makes it feel like trivia to memorize. It isn't. It's the difference between a product that degrades gracefully and one that falls over completely the first time it gets popular.

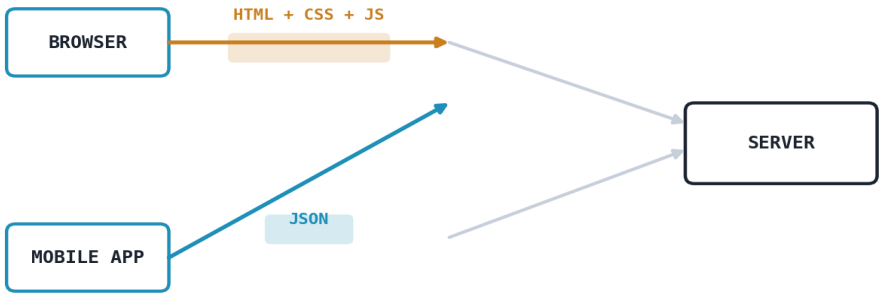
This first part stays close to the ground: what a request actually is, where it comes from, and what happens the moment more than one person sends one at the same time. Parts two through four build caching, asynchronous messaging, and global distribution on top of this foundation.

01

Where does traffic even come from?

Every request hits your server from one of two doors — a browser or a mobile app — and the two doors want completely different things.

- **Browser:** wants the full page — HTML, CSS and JS, login via cookies, assumes a stable connection, always shows the latest version on refresh.
- **Mobile app:** already has the layout installed, just wants raw JSON, uses token-based auth (JWT), has to support multiple app versions at once, and assumes a flaky connection.



Browser sends/receives full page payloads · Mobile app sends/receives lightweight JSON

02

Splitting the load: web tier vs. data tier

This setup works for a modest user base, but breaks under real traffic because not everything strains at the same rate.

Web tier handles incoming requests and returns HTML or JSON. Add two or three more servers behind a load balancer when traffic spikes cheap, fast, repeatable.

Data tier is the database. Scaling it means read replicas and sharding, not just "add a box." Keeping the tiers separate means the web layer grows freely without ever touching the database.

Web tier scales out freely · Data tier is scaled carefully

03

Choosing a database: relational vs. non-relational

The honest answer to "which database" is: it depends on whether your data wants to be a spreadsheet or a folder.

Relational (PostgreSQL, MySQL, SQLite, Oracle) is a perfectly organized spreadsheet fixed tables, fixed columns, clear relationships. That rigidity buys strong consistency, ideal for a bank balance or checkout flow.

Non-relational (Cassandra, Redis, Neo4j, MongoDB) is an open folder documents, key-value pairs, graphs, whatever shape the data has. No rigid schema, built to scale horizontally with low latency.

Rule of thumb: fixed shape + 100% accuracy → relational. Flexible shape + huge volume + speed → non-relational.

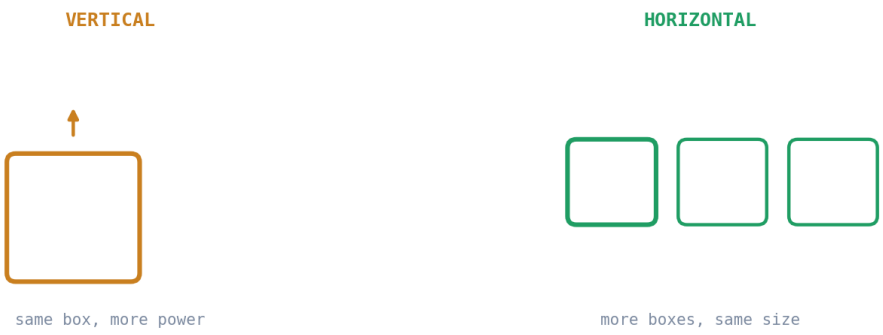
04

Scaling up vs. scaling out

"Code is easy. Keeping it alive when a million people show up at once that's the actual job." Two ways to respond.

Vertical scaling turns your old computer into a superhero same machine, bigger engine. Almost no code changes, but there's a hard ceiling, and one machine is still a single point of failure.

Horizontal scaling buys more ordinary cars instead of one giant bus. Scalability becomes close to infinite, and losing one server doesn't take the system down but now something has to divide the traffic between them.



Vertical: same box, more power · Horizontal: more boxes, same size

---

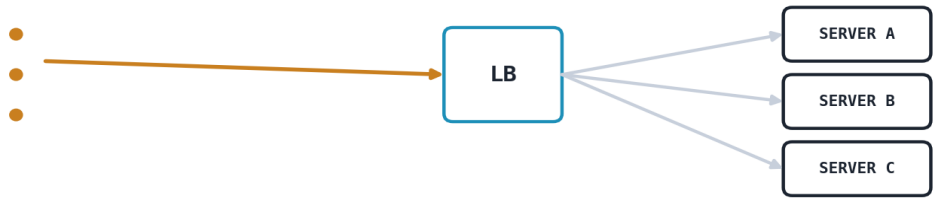
05

The load balancer: traffic's police officer

10,000 people hit your site at once, and you're running three servers. Someone has to direct traffic.

It sits at the very front and hands off every request using one of a few algorithms: **Round Robin** (straight down the line), **Least Connections** (busiest servers get skipped), or **IP Hash** (same user always lands on the same server).

It also runs constant health checks. Common software options are Nginx and HAProxy; F5 is a well-known hardware option; cloud providers like AWS will run it for you if you'd rather not configure it yourself.



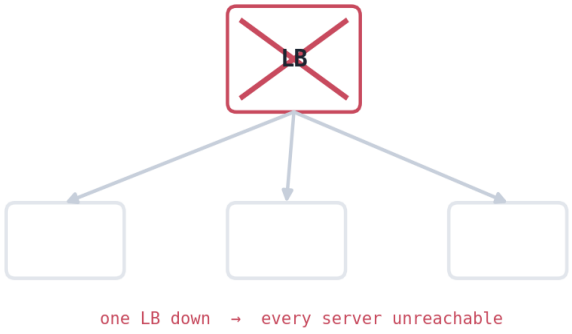
Incoming requests, distributed evenly across servers A, B, C

06

The hidden cracks: single points of failure

Everything depends on that one load balancer, and every server depends on that one database. Kill either, and the whole product goes down.

A single point of failure (SPOF) is any one component that, if it disappears, takes the entire system with it. The fix isn't a clever algorithm it's refusing to let anything important exist in a quantity of one.



When the only load balancer fails, every healthy server becomes unreachable

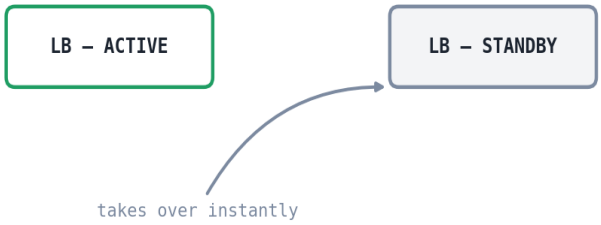
---

07

Redundancy: always have a backup engine

A commercial airplane always flies with two engines. Apply the same logic to your load balancer.

Run two load balancers instead of one. Traffic flows to the active one as normal; the passive one watches and waits. The moment the active balancer crashes, the standby takes over instantly no one paged at 3am.



Primary fails → standby takes over

08

Health checks: the heartbeat monitor

Having a backup is useless if nothing knows when to use it.

This is exactly like an ICU heart-rate monitor: a steady beep confirming the patient is stable, and an instant alarm the second something changes. Load balancers ping every server on a tight loop, quietly removing any that stop answering.

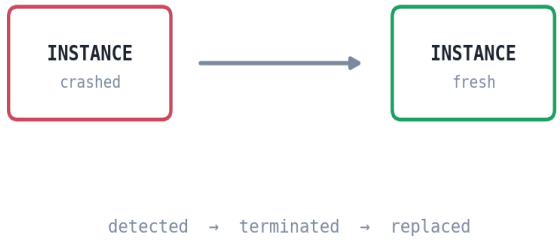
Continuous pings confirm every node is still alive

09

Self-healing systems: the lizard's tail

When a lizard loses its tail, it doesn't wait for a vet it just grows another one.

The moment a health check flags a dead server, automation (AWS Auto Scaling and similar tools) terminates the broken instance and spins up a fresh one in its place — usually within seconds, no engineer touching a keyboard.



Failed instance detected → terminated → replaced automatically

---

10

Putting it together: a Friday night on Netflix

Every idea above, happening in under a second, while you're mid-episode and don't notice a thing.

Redundancy meant a backup server was already running. The health check caught the failure in a fraction of a second and rerouted the stream before any buffering. Self-healing quietly rebuilt the failed server in the background, so capacity never shrank. That's not luck — that's the design holding under exactly the kind of pressure it was built for.

Server crashes → health check detects → stream rerouted → new server spun up

---

Coming in Part 2

Caching, queues, and the art of not asking the database twice. Now that requests can survive a crash, the next problem is speed and it starts with everything in this post hitting the database far more than it needs to.